

1. T O

2. Przebieg procesu kompilacji dla metody starej (Screeny 1,2,3,4,5,6)

2.1 Etapy procesu

2.1.1 Proces przebiegł bezproblemowo oraz dość szybko. Jedyne zastrzeżenie jest przy wyborze modułów, które wymagało manualnego potwierdzenia bądź odrzucenia.

3. Przebieg procesu kompilacji dla metody nowej (Screeny 11,12,13,14)

3.1 Etapy procesu

3.1.1 Proces przebiegł bezproblemowo oraz dość szybko.

4. Wnioski

4.1 Porównanie metod konfiguracji (Stara vs. Nowa)

4.1.1 Metoda stara (make localmodconfig): Jest to wbudowany w system budowania jądra mechanizm automatycznego odchudzania konfiguracji. Narzędzie to analizuje aktywne sterowniki w systemie i wyłącza wszystko, co zbędne. Podczas generowania pliku .config system poprosił o interaktywne zatwierdzenie nowych opcji (wprowadzonych w wersji jądra 7.0.9 względem jądra hosta), co wymagało ręcznego potwierdzenia domyślnych wartości.

4.1.2 Metoda nowa (streamline_config.pl): Jest to bezpośrednie wywołanie skryptu w języku Perl, który de facto stanowi silnik stojący za poleceniem localmodconfig. Samodzielne uruchomienie skryptu pozwala na lepszą kontrolę nad strumieniem wyjściowym i pozwala na jawne nadpisanie pliku tymczasowego, co po wykonaniu make olddefconfig automatycznie akceptuje domyślne ustawienia dla nowych flag architektonicznych, eliminując potrzebę interaktywnego odpowiadania na pytania.

4.2 Wniosek: Obie metody dążą do tego samego rezultatu – drastycznego zmniejszenia rozmiaru finalnego obrazu jądra (bzImage) i przyspieszenia kompilacji poprzez odrzucenie tysięcy uniwersalnych, nieużywanych sterowników dystrybucyjnych.

4.3 Optymalizacja procesu kompilacji i wydajność

4.3.1 Kompilacja została przeprowadzona na procesorze AMD Ryzen 7 5700X przy użyciu flagi -j\$(nproc) (wykorzystanie pełnych 16 wątków procesora).

4.3.2 Wydajność sprzętowa: Procesor desktopowy, w przeciwieństwie do jednostek mobilnych (np. z architekturą rdzeni P/E, takich jak i5-1235u), charakteryzuje się brakiem dławienia termicznego (thermal throttling) i jednorodną architekturą rdzeni, co przełożyło się na błyskawiczne ukończenie kompilacji.

4.3.3 Liczba wątków: Zgodnie z sugestią dotyczącą teoretycznej zasady stosowania mnożnika $1.5 \times \text{liczba wątków}$ (w celu niwelowania przestojów I/O), w nowoczesnym środowisku wyposażonym w szybkie dyski NVMe oraz technologię SMT (Hyper-Threading), optymalnym podejściem okazało się idealne dopasowanie liczby zadań do liczby fizycznie zgłaszanych wątków systemowych. Zapobiegło to narzutowi związanemu z częstym przełączaniem kontekstu (context switching) przez planistę jądra.